

OpenCV Image Beautification

1. Experiment Objective

Master image enhancement techniques: brightness/contrast adjustment, image restoration (inpainting), and skin beautification (smoothing).

2. Experiment Principle

1. Keyword Explanation

Keyword	Description
Brightness enhancement	Adds a fixed offset (or multiplies by a coefficient) to the BGR values of each pixel, making the overall image brighter
Image restoration (inpainting)	Uses interpolation algorithms to automatically fill in missing regions based on the pixel information around the damaged area
INPAINT_TELEA	FMM (Fast Marching Method) restoration algorithm that advances inward step by step from the damaged boundary, filling with the weighted average of neighboring pixels
Skin beautification (smoothing)	Enhances the blue and green channels (while weakening red), giving the skin a cool white complexion

2. Technical Architecture

This experiment is a pure offline Python script. Image beautification operations are divided into two categories:

- **Global pixel operations** (brightness enhancement, skin beautification): iterate over each pixel and perform arithmetic operations on its BGR channel values
- **Region restoration** (inpainting): create a single-channel mask image marking the damaged regions, then call the `cv2.inpaint()` algorithm to fill them automatically

Data flow: local image → `cv2.imread()` → pixel-level operations / mask-guided restoration → `cv2.imshow()` side-by-side display.

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	☒ Supported
No-PTZ version	☒ Supported
Required peripherals	None (local image processing, no need to start the car)

2. Startup Method

```
# Brightness enhancement
python3
~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/opencv_image_enhancement/o
pencv_image_brightness_enhancement.py

# Generate the damaged image (running this creates damaged.png, which the repair
mode depends on)
python3
~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/opencv_image_enhancement/o
pencv_create_damaged_image.py

# Repair the damaged image
python3
~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/opencv_image_enhancement/o
pencv_repair_damaged_images.py

# Skin beautification
python3
~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/opencv_image_enhancement/o
pencv_skin_smoothing_brightening.py
```

Note: the `repair` mode depends on the `damaged.png` file generated by running `create_damaged_image.py` first. Please run `damaged` first and then `repair`.

3. Operation Instructions

- `opencv_image_brightness_enhancement.py`: increases brightness and contrast
- `opencv_create_damaged_image.py`: draws a black cross on the original image to simulate damage, and saves it with `cv2.imwrite()`
- `opencv_repair_damaged_images.py`: reads the damaged image and automatically repairs it with `cv2.inpaint()`
- `opencv_skin_smoothing_brightening.py`: enhances the B and G channels to achieve skin beautification

Press `q` to exit.

4. Stop Method

Press `q` to close all OpenCV windows and exit the program.

4. Experiment Phenomena

- Master the use of the `cv2.inpaint()` image restoration API
- Understand the image enhancement principle of pixel-level BGR channel operations

Original image: The input image for image beautification; all enhancement and restoration operations are compared against this image.



Brightness enhancement: Adds 100 to each of the BGR channels of every pixel (capped at 255), significantly increasing the overall brightness of the image.



Damaged image: Black crosses are artificially drawn on the image to simulate scratches and occlusion, used to test the restoration algorithm.



Restoration result: Uses `cv2.inpaint()` to automatically fill the damaged regions of the image based on neighboring pixels, restoring the original picture.



Skin beautification: Enhances the blue ($\times 1.4+5$) and green ($\times 1.4+5$) channels while keeping the red channel unchanged, making the skin tone fairer.



5. Program Analysis

Source code path:

- Script directory:
`~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/opencv_image_enhancement/`
- Test image:
`~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/images/yahboom.png`
- Damaged image:
`~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/images/damaged.png`

1. Core Logic Analysis

The core of image beautification is direct arithmetic on BGR pixel values. Below is the per-pixel processing for brightness and skin tone, as well as the mask-guided flow of image restoration.

Brightness enhancement (opencv_image_brightness_enhancement.py)

```
# Source file:
~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/opencv_image_enhancement/opencv_image_brightness_enhancement.py
import cv2
import numpy as np

if __name__ == '__main__':
    img =
cv2.imread('~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/images/yahboom.png', cv2.IMREAD_UNCHANGED)
    if img is None:
        print("Image read failed, please check that the images/yahboom.png path is correct")
        exit()

    imgInfo = img.shape
    height = imgInfo[0]
    width = imgInfo[1]

    dst = np.zeros((height, width, 3), np.uint8)

    for i in range(0, height):
        for j in range(0, width):
```

```

        b, g, r = img[i, j]

        bb = int(b) + 100
        gg = int(g) + 100
        rr = int(r) + 100

        if bb > 255:
            bb = 255
        if gg > 255:
            gg = 255
        if rr > 255:
            rr = 255

        dst[i, j] = (bb, gg, rr)

while True:
    cv2.imshow("yahboom_brightness_enhancement", dst)
    action = cv2.waitKey(10) & 0xFF
    if action == ord('q') or action == 113:
        break

cv2.destroyAllWindows()

```

Key logic notes:

- Iterates over every pixel (a `height x width` double loop), adding 100 to the B, G, and R channels of each pixel
- Each channel value is clipped to a maximum of 255 (`uint8` range is 0~255; exceeding it will overflow and wrap around)
- Why not use `cv2.convertScaleAbs(img, alpha=1, beta=100)`? This script demonstrates the underlying principle so that readers can understand how each pixel changes

Image restoration (opencv_repair_damaged_images.py)

```

# Source file:
~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/opencv_image_enhancement/opencv_repair_damaged_images.py
import cv2
import numpy as np

if __name__ == '__main__':
    dam_img =
cv2.imread('~microros_ws/src/microros_v2_opencv_base_pkg/scripts/images/damaged.png', cv2.IMREAD_UNCHANGED)
    if dam_img is None:
        print("Image read failed, please check that the images/damaged.png path is correct")
        exit()

    imgInfo = dam_img.shape
    height = imgInfo[0]
    width = imgInfo[1]

    # Create a single-channel mask image
    paint = np.zeros((height, width, 1), np.uint8)

    for i in range(50, 100):

```

```

    paint[i, 50] = 255
    paint[i, 50 + 1] = 255
    paint[i, 50 - 1] = 255

    for i in range(100, 150):
        paint[150, i] = 255
        paint[150 + 1, i] = 255
        paint[150 - 1, i] = 255

    # Repair the image
    dst_img = cv2.inpaint(dam_img, paint, 3, cv2.INPAINT_TELEA)

    while True:
        cv2.imshow("dam_img", dam_img)
        cv2.imshow("paint", paint)
        cv2.imshow("dst", dst_img)
        action = cv2.waitKey(10) & 0xFF
        if action == ord('q') or action == 113:
            break

    cv2.destroyAllWindows()

```

Key logic notes:

- **Mask image:** a single-channel `uint8` image where non-zero values mark the pixels to be repaired. Two white lines (value 255) mark the vertical scratch ($x=50, y=50\sim99$) and the horizontal scratch ($y=150, x=100\sim149$) respectively
- `cv2.inpaint(src, inpaintMask, inpaintRadius, flags):`
 - `inpaintRadius=3`: when restoring, references the neighborhood information within a 3-pixel radius around each damaged pixel
 - `cv2.INPAINT_TELEA`: the FMM algorithm, advancing inward along the damaged boundary; suitable for thin scratches. The alternative `INPAINT_NS` is based on the Navier-Stokes equation and is suitable for larger-area filling

Skin beautification (opencv_skin_smoothing_brightening.py)

```

# Source file:
~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/opencv_image_enhancement/opencv_skin_smoothing_brightening.py
import cv2
import numpy as np

if __name__ == '__main__':
    img =
    cv2.imread('~microros_ws/src/microros_v2_opencv_base_pkg/scripts/images/yahboom.png', cv2.IMREAD_UNCHANGED)
    if img is None:
        print("Image read failed, please check that the images/yahboom.png path is correct")
        exit()

    imgInfo = img.shape
    height = imgInfo[0]
    width = imgInfo[1]

    dst = np.zeros((height, width, 3), np.uint8)

```

```

for i in range(0, height):
    for j in range(0, width):
        b, g, r = img[i, j]

        bb = int(b * 1.4) + 5
        gg = int(g * 1.4) + 5

        if bb > 255:
            bb = 255
        if gg > 255:
            gg = 255

        dst[i, j] = (bb, gg, r)

while True:
    cv2.imshow("origin", img)
    cv2.imshow("dst", dst)
    action = cv2.waitKey(10) & 0xFF
    if action == ord('q') or action == 113:
        break

cv2.destroyAllWindows()

```

Key logic notes:

- Only the **B (blue) and G (green)** channels are enhanced (coefficient 1.4 + offset 5), while the **R (red) channel stays unchanged**
- In BGR color space, skin tone contains a relatively large red component. Weakening red's relative proportion = boosting cool tones = making the skin appear fairer
- The difference from brightness enhancement: brightness enhancement increases all three channels equally (preserving hue), while skin beautification is selective enhancement (changing hue)

2. Key Parameters

Parameter	Type	Default	Description
Brightness offset	int	100	Increment for each BGR channel; the larger the value, the brighter the image (capped at 255)
Skin B/G coefficient	float	1.4	Multiplier for the B and G channels; setting it to 1.0 means no enhancement
<code>cv2.inpaint</code> 's <code>inpaintRadius</code>	int	3	Restoration reference radius; too large blurs details, too small under-restores
<code>cv2.INPAINT_TELEA</code>	flag	—	FMM restoration algorithm; faster than NS and suitable for thin scratches

3. Node Notes

This experiment is a pure Python script and does not use ROS2 nodes.

6. Common Problems

Problem	Cause	Solution
Some areas are overexposed after brightness enhancement	The offset is too large, causing many pixels to hit the 255 cap	Reduce the offset or use <code>cv2.convertScaleAbs()</code> for adaptive adjustment
Poor image restoration results	inpaintRadius set improperly	For thin scratches use radius=3~5; for large damaged areas use radius=5~10
The image turns bluish/greenish after skin beautification	B and G channel coefficients are too high	Lower the coefficients to 1.1~1.2, or compensate with the R channel after enhancement
The repair mode reports an image load failure	The damaged image was not generated by running the damaged mode first	Run <code>opencv_create_damaged_image.py</code> first, then the repair script